



New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

Dimensionality Reduction

Islam Reda

Associate Professor

islam-cis@mans.edu.eg



Outline

- The Curse of Dimensionality
- Main Approaches for Dimensionality Reduction
 - Projection
 - Manifold Learning
- Principal Component Analysis (PCA)
- Kernel PCA
- Locally Linear Embedding (LLE)



Dimensionality Reduction

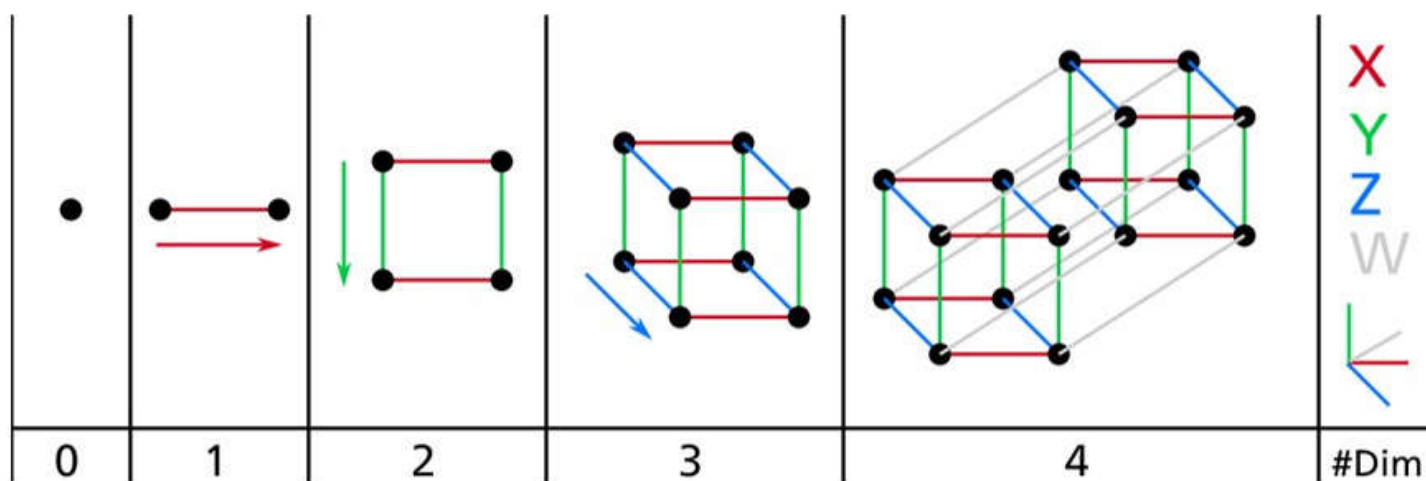
- Many Machine Learning problems involve thousands or even millions of features for each training instance.
- Not only does this make **training extremely slow**, it can also make it **much harder to find a good solution**. This problem is often referred to as the **curse of dimensionality**.
- Fortunately, it is often possible to **reduce the number of features** considerably.



Dimensionality Reduction

- Reducing dimensionality does **lose some information**, so even though it will speed up training, it may also make your **system perform slightly worse**.
- Two main benefits:
 - **Speeding up training**
 - Useful for **data visualization**

The Curse of Dimensionality



- **High-dimensional datasets** are at risk of being **very sparse**: most **training instances** are likely to be **far away** from each other. A **new instance** will likely be **far away** from any **training instance**.



Main Approaches for Dimensionality Reduction

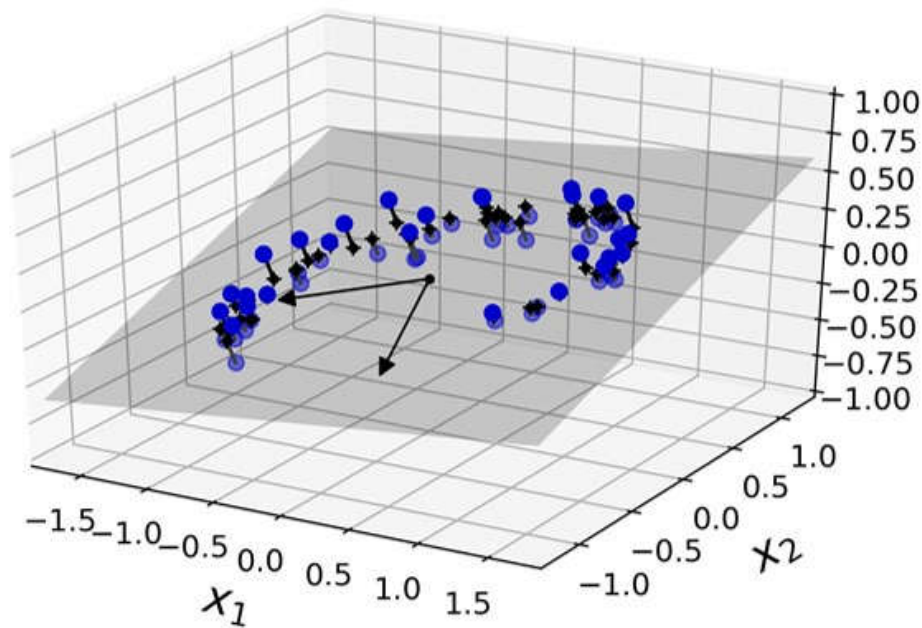
- Two main approaches to reducing dimensionality:
 - Projection
 - Manifold Learning



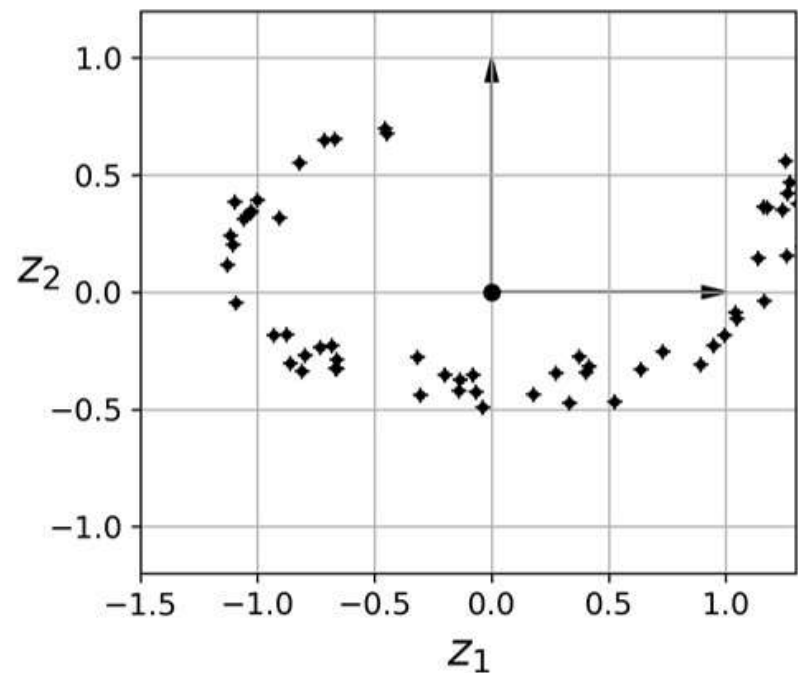
Projection

- Training instances are not spread out uniformly across all dimensions. Many features are almost constant, while others are highly correlated.
- As a result, **all training instances** actually **lie within** (or close to) a **much lower-dimensional subspace** of the high-dimensional space.
- If we project every training instance perpendicularly onto this subspace, we get a lower-dimensional dataset.

Projection



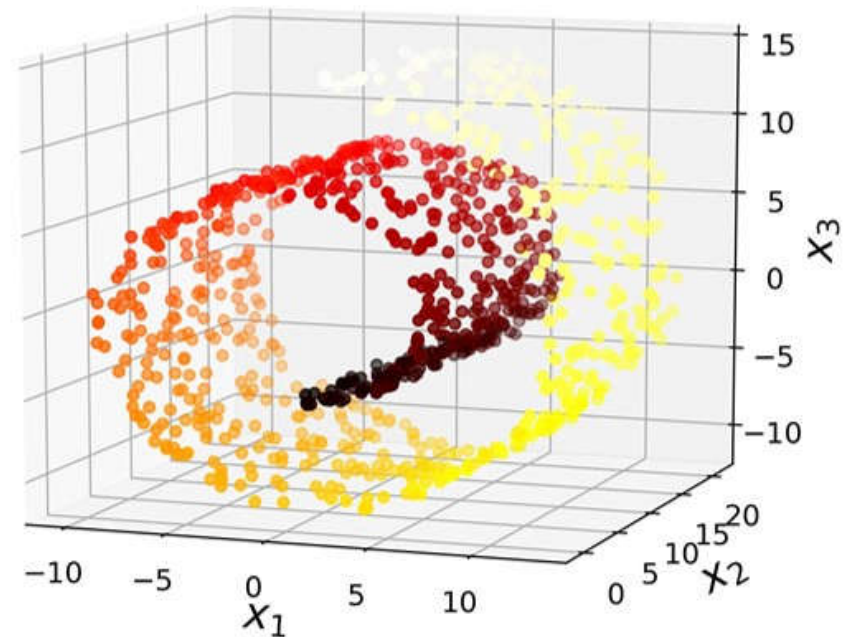
A 3D dataset lying close to a 2D subspace



The new 2D dataset after projection

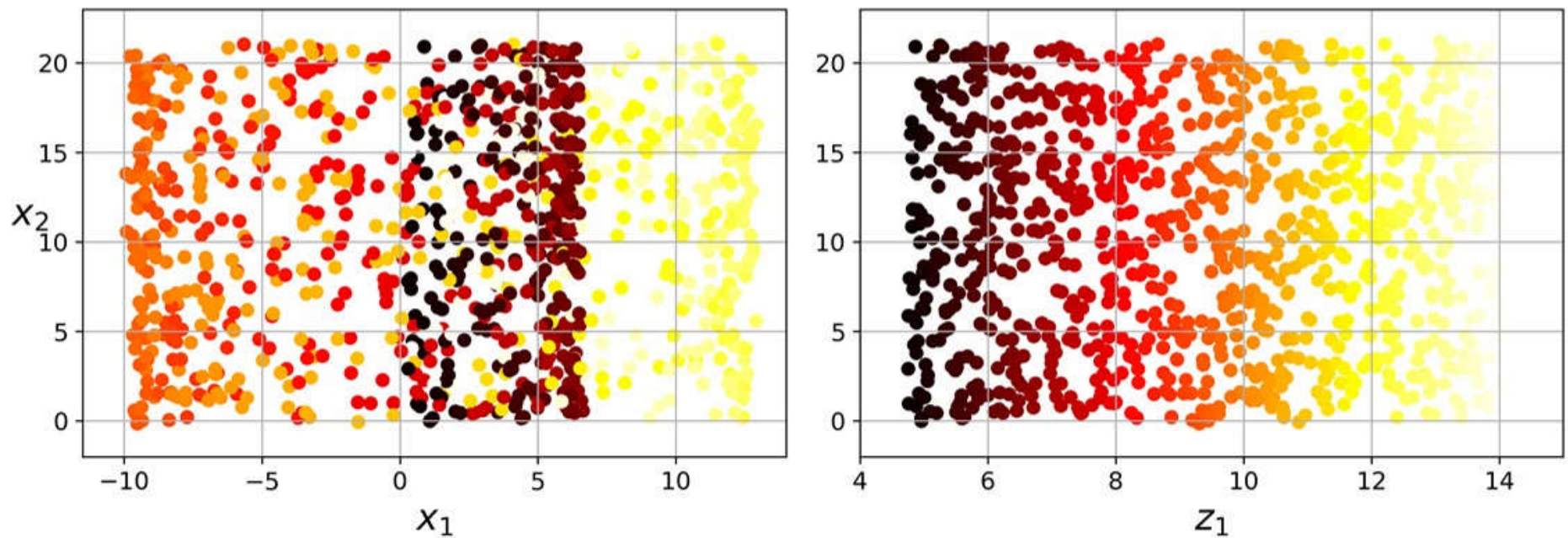
Projection

- However, projection is **not always the best approach** to dimensionality reduction. In many cases the **subspace may twist and turn**, such as in the famous **Swiss roll toy dataset**.



Swiss roll dataset

Projection



Squashing by projecting onto a plane (left) versus unrolling the Swiss roll (right)



Manifold Learning

- The Swiss roll is an example of a **2D manifold**. A 2D manifold is a **2D shape that can be bent and twisted in a higher-dimensional space**.
- Many dimensionality reduction algorithms work by **modeling the manifold on which the training instances lie**; this is called **Manifold Learning**.
- It relies on the **manifold assumption (manifold hypothesis)**: most real-world high-dimensional datasets lie close to a much lower-dimensional manifold.



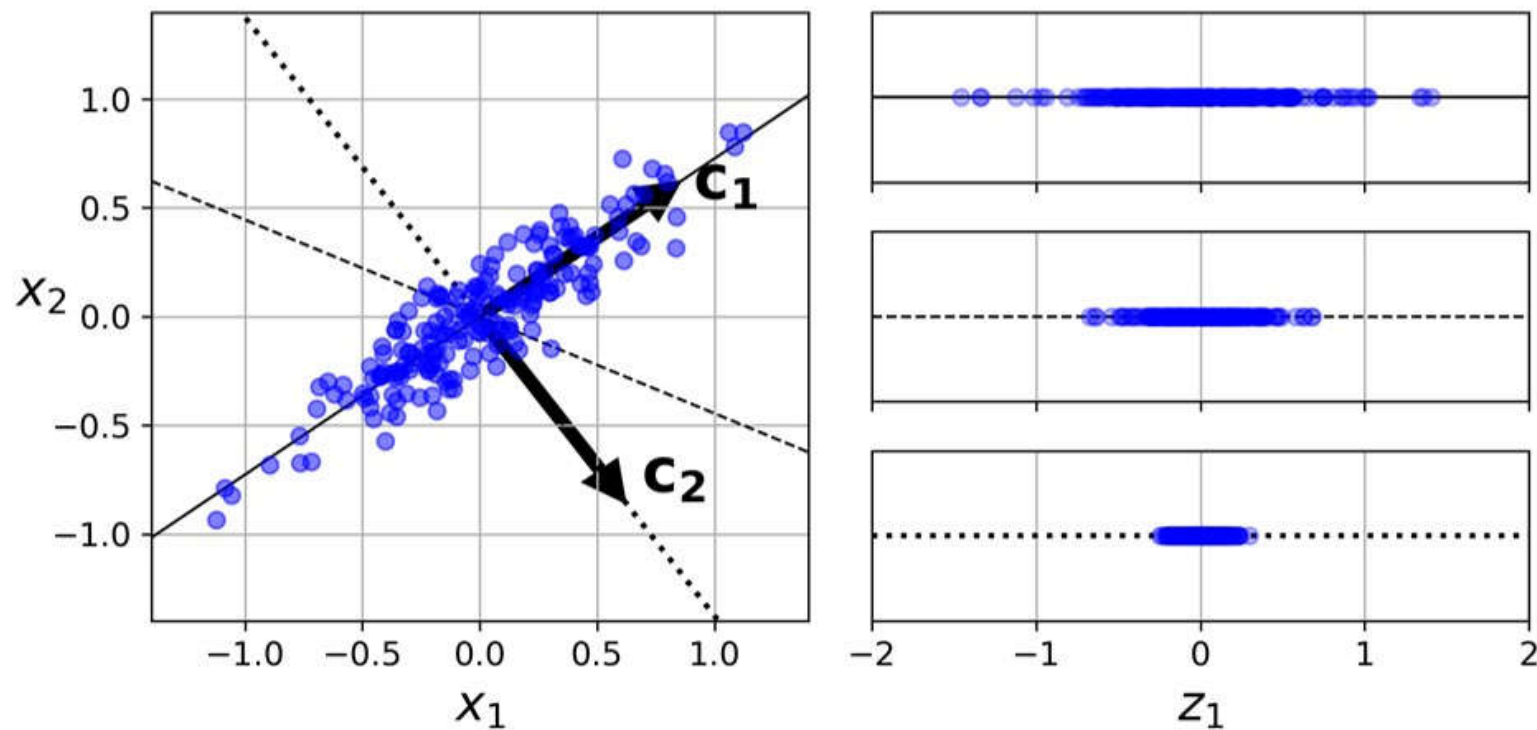
Principal Component Analysis (PCA)

- **PCA** is by far the **most popular** dimensionality reduction algorithm.
- First it **identifies the hyperplane** that lies **closest to the data**.
- Then, it **projects the data onto it**.

Preserving the Variance

- Before you can project the training set onto a lower-dimensional hyperplane, you first need to **choose the right hyperplane**.
- The **hyperplane that preserves the maximum amount of variance**, as it will most likely **lose less information** than the other projections.

Preserving the Variance



Selecting the subspace onto which to project



Principal Components

- PCA identifies the **axis that accounts for the largest amount of variance** in the training set.
- It also finds a **second axis**, orthogonal to the first one, **that accounts for the largest amount of remaining variance**.
- The **unit vector** that defines the i^{th} axis is called the i^{th} **principal component (PC)**.
- So how can you find the principal components of a training set?
- **Singular Value Decomposition (SVD)** can decompose the training set matrix $\mathbf{X}$ into the matrix multiplication of three matrices $\mathbf{U} \Sigma \mathbf{V}^T$, where $\mathbf{V}$ contains all the principal components.



Principal Components

$$\mathbf{V} = \begin{pmatrix} | & | & | \\ c_1 & c_2 & \dots & c_n \\ | & | & | \end{pmatrix}$$

- This code uses NumPy's `svd()` function to obtain all the principal components of the training set, then extracts the first two PCs:

```
X_centered = X - X.mean(axis=0)
U, s, Vt = np.linalg.svd(X_centered)
c1 = Vt.T[:, 0]
c2 = Vt.T[:, 1]
```

- PCA assumes that the dataset is centered around the origin.



Projecting Down to d Dimensions

- To project the training set onto the hyperplane, you can simply compute the matrix multiplication of the training set matrix $\mathbf{X}$ by the matrix $\mathbf{W}_d$, defined as the matrix containing the first d principal components

$$\mathbf{X}_{d\text{-proj}} = \mathbf{X}\mathbf{W}_d$$

- The following Python code projects the training set onto the plane defined by the first two principal components:

```
W2 = Vt.T[:, :2]  
X2D = X_centered.dot(W2)
```



PCA Using Scikit-Learn

- The following Python code applies PCA to reduce the dimensionality of the dataset down to two dimensions:

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components = 2)  
X2D = pca.fit_transform(X)
```

- After fitting the PCA transformer to the dataset, you can **access the principal components** using the `components_` variable (note that it contains the PCs as horizontal vectors, so, the **first PC** is equal to `pca.components_.T[:,0]`)



Explained Variance Ratio

- The **explained variance ratio** of each principal component is available via the `explained_variance_ratio_` variable.
- It indicates the proportion of the dataset's variance that lies along the axis of each principal component.

```
>>> pca.explained_variance_ratio_  
array([0.84248607, 0.14631839])
```

- 84.2% of the dataset's variance lies along the first axis, and 14.6% lies along the second axis.



Choosing the Right Number of Dimensions

- It is generally preferable to choose the number of dimensions that add up to a sufficiently large portion of the variance (e.g., 95%).
- Instead of specifying the number of principal components you want to preserve, you can set **n_components** to be a **float between 0.0 and 1.0**, indicating the **ratio of variance you wish to preserve**:

```
pca = PCA(n_components=0.95)  
X_reduced = pca.fit_transform(X_train)
```

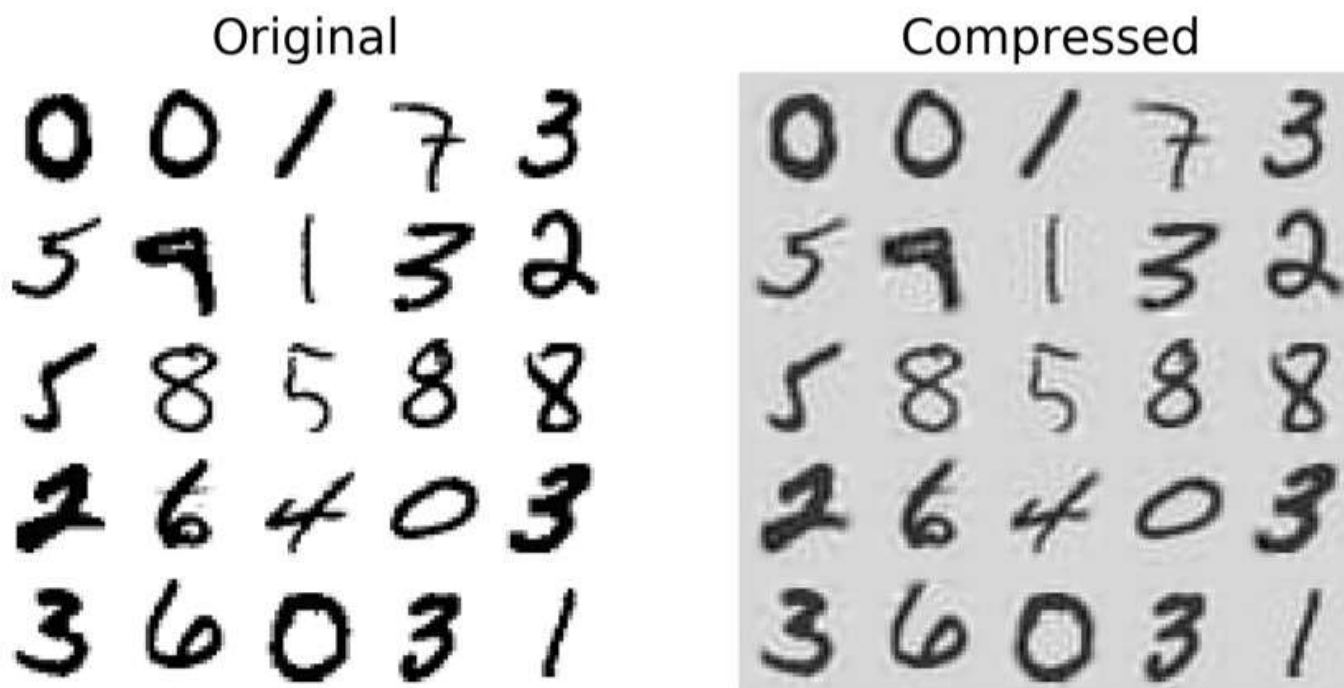


PCA for Compression

- Applying PCA to the MNIST dataset while preserving 95% of its variance results in each instance having just over 150 features, instead of the original 784 features. The **dataset is now less than 20% of its original size!** This is a **reasonable compression ratio**.
- It is also possible to decompress the reduced dataset back to 784 dimensions by applying the **inverse transformation of the PCA** projection.
- The mean squared distance between the original data and the reconstructed data is called the **reconstruction error**.

```
pca = PCA(n_components = 154)
X_reduced = pca.fit_transform(X_train)
X_recovered = pca.inverse_transform(X_reduced)
```

PCA for Compression



MNIST compression preserving 95% of the variance



Incremental PCA

- One problem with the preceding implementations of PCA is that they **require the whole training set to fit in memory** in order for the algorithm to run.
- **Incremental PCA (IPCA)** algorithms **split the training set into mini-batches** and feed an IPCA algorithm one mini-batch at a time.

```
from sklearn.decomposition import IncrementalPCA
```

```
n_batches = 100
```

```
inc_pca = IncrementalPCA(n_components=154)
```

```
for X_batch in np.array_split(X_train, n_batches):  
    inc_pca.partial_fit(X_batch)
```

```
X_reduced = inc_pca.transform(X_train)
```



Kernel PCA

- The **kernel trick** is a mathematical technique that implicitly maps instances into a very high-dimensional space (called the feature space), enabling nonlinear classification and regression with Support Vector Machines.
- The same trick can be applied to PCA, making it possible to **perform complex nonlinear projections** for dimensionality reduction.

```
from sklearn.decomposition import KernelPCA
```

```
rbf_pca = KernelPCA(n_components = 2, kernel="rbf", gamma=0.04)  
X_reduced = rbf_pca.fit_transform(X)
```



Selecting a Kernel and Tuning Hyperparameters

- As **kPCA** is an **unsupervised learning algorithm**, there is no obvious performance measure to help you select the best kernel and hyperparameter values.
- However, dimensionality reduction is often a **preparation step for a supervised learning task** (e.g., classification), so you can simply use grid search to select the kernel and hyperparameters that lead to the best performance on that task.

```
from sklearn.model_selection import GridSearchCV  
from sklearn.linear_model import LogisticRegression  
from sklearn.pipeline import Pipeline
```



Selecting a Kernel and Tuning Hyperparameters

```
clf = Pipeline([
    ("kpca", KernelPCA(n_components=2)),
    ("log_reg", LogisticRegression())
])

param_grid = [{
    "kpca__gamma": np.linspace(0.03, 0.05, 10),
    "kpca__kernel": ["rbf", "sigmoid"]
}]

grid_search = GridSearchCV(clf, param_grid, cv=3)
grid_search.fit(X, y)

>>> print(grid_search.best_params_)
{'kpca__gamma': 0.043333333333333335, 'kpca__kernel': 'rbf'}
```



Locally Linear Embedding (LLE)

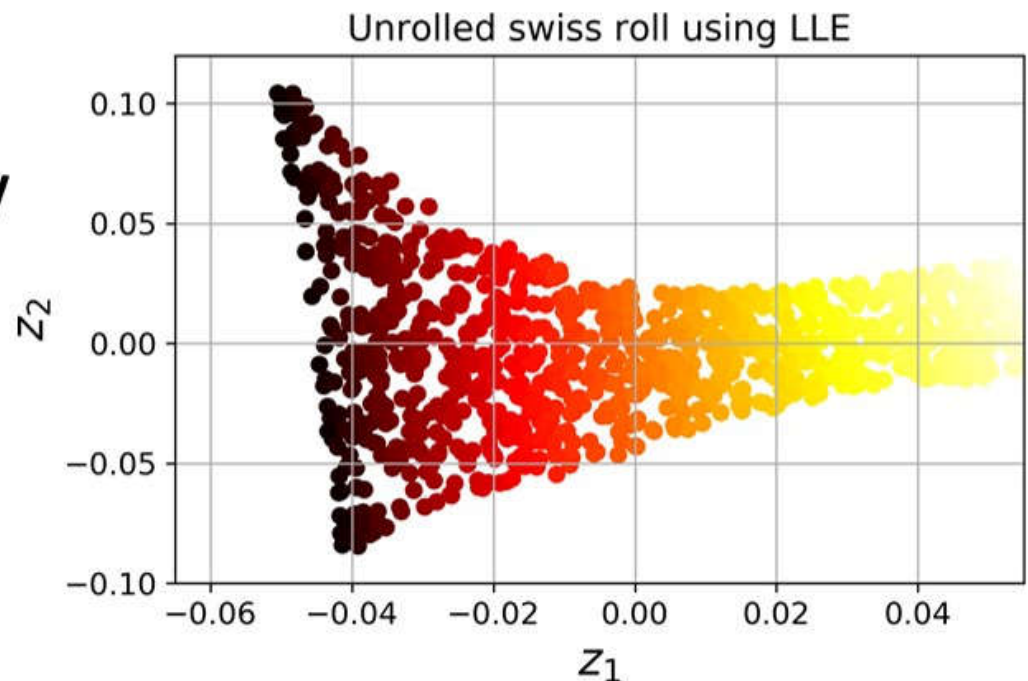
- **LLE is a Manifold Learning technique** that does not rely on projections like the previous algorithms.
- LLE works by **first measuring how each training instance linearly relates to its closest neighbors (c.n.)**.
- Then, **looking for a low-dimensional representation** of the training set **where these local relationships are best preserved**.
- The following code uses Scikit-Learn's **LocallyLinearEmbedding** class to unroll the Swiss roll.

```
from sklearn.manifold import LocallyLinearEmbedding
```

```
lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10)  
X_reduced = lle.fit_transform(X)
```

Locally Linear Embedding (LLE)

- The Swiss roll is completely unrolled and **the distances between instances are locally well preserved**.
- Due to its **computational complexity**, LLE scales poorly to very large datasets.



Unrolled Swiss roll using LLE



Thank you